Lec13

Functions in Javascript

A **function** is a reusable block of code designed to perform a specific task. Functions help organize code, avoid repetition, and improve maintainability.

Why Functions are Important

Without functions:

```
let a = 10;
let b = 20;
console.log(a + b);

let c = 30;
let d = 40;
console.log(c + d);
```

The same logic is repeated multiple times.

Using functions:

```
function add(x, y) {
   return x + y;
}

console.log(add(10, 20));
console.log(add(30, 40));
```

The code becomes reusable and easier to maintain.

Function Declaration

A function declaration uses the `function` keyword.

```
function greet() {
   console.log("Welcome to JavaScript");
}

greet();
```

Structure

```
function functionName(parameters) {
   // statements
   return value;
}
```

Example

```
function multiply(a, b) {
   return a * b;
}

console.log(multiply(5, 4));
```

Function Parameters and Arguments

Parameters

Variables defined in the function definition.

```
function greet(name) {
   console.log("Hello " + name);
}
```

`name` is a parameter.

Arguments

Actual values passed to the function.

greet("Naman");

"Naman" is an argument.

Function Expression

A function can be stored inside a variable.

```
const greet = function() {
   console.log("Hello");
};

greet();
```

Unlike function declarations, function expressions are not fully hoisted.

Anonymous Function

Functions without names.

```
const sayHello = function() {
   console.log("Hello");
};
```

Anonymous functions are commonly used as callbacks.

Return Statement

The `return` keyword sends a value back to the caller.

```
function square(num) {
   return num * num;
}

let result = square(5);
console.log(result);
```

Output:

25

Higher-Order Functions

A **Higher-Order Function** is a function that:

1. Accepts another function as an argument.
2. Returns a function.

This concept is one of the foundations of functional programming in JavaScript.

Function as an Argument

```
function greet(name) {
  return "Hello " + name;
}

function processUser(callback) {
  console.log(callback("Naman"));
}

processUser(greet);
```

Output:

Hello Naman

Here:

- `processUser()` is a Higher-Order Function.
- `greet()` is a callback function.

Function returning another Function

```
function multiplier(factor) {
   return function(number) {
       return number * factor;
   };
}

const double = multiplier(2);

console.log(double(10));
```

Output:

20

The returned function remembers the value of `factor`.

Real-life Analogy

```
function multiplier(factor) {
   return function(number) {
       return number * factor;
   };
}

const double = multiplier(2);

console.log(double(10));
```

Output:

20

The returned function remembers the value of `factor`.

Common High Order Function

map()

Transforms every element.

```
const numbers = [1, 2, 3, 4];

const squares = numbers.map(function(num) {
   return num * num;
});

console.log(squares);
```

Output:

[1, 4, 9, 16]

filter()

Selects elements based on a condition.

```
const numbers = [1, 2, 3, 4, 5];

const even = numbers.filter(function(num) {
   return num % 2 === 0;
});

console.log(even);
```

Output:

[2, 4]

reduce()

Reduces an array to a single value.

```
const numbers = [1, 2, 3, 4];

const total = numbers.reduce(function(sum, num) {
   return sum + num;
}, 0);

console.log(total);
```

Output:

10

Lambda Functions (Arrow Functions)

Arrow Functions were introduced in ES6 to provide a shorter syntax for writing functions.

They are often called **Lambda Functions** because they resemble lambda expressions used in functional programming languages.

Traditional Function

```
function add(a, b) {
  return a + b;
}
```

Arrow Function

```
const add = (a, b) => {
  return a + b;
};
```

Short Form

```
const add = (a, b) => a + b;
```

No braces and no `return` keyword needed.

Single Parameter

```
const square = x => x * x;
```

No Parameters

```
const hello = () => "Hello World";
```

Arrow Functions with Arrays

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(num => num * 2);

console.log(doubled);
```

Difference between normal and arrow function

Normal Function

```
const person = {
   name: "Naman",

   greet: function() {
       console.log(this.name);
   }
};

person.greet();
```

Output:

Naman

Arrow Function

```
const person = {
   name: "Naman",

   greet: () => {
       console.log(this.name);
   }
};

person.greet();
```

Output:

undefined

Arrow functions do not have their own `this`.

Hoisting

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before execution.

Variable Hoisting with var

```
console.log(age);

var age = 25;
```

JavaScript internally treats it as:

```
var age;

console.log(age);

age = 25;
```

Output:

```
undefined
```

Let and const Hoisting

```
console.log(age);

let age = 25;
```

Output:

ReferenceError

Temporal Dead Zone (TDZ)

The time between variable declaration and initialization is called the **Temporal Dead Zone**.

```
{
  console.log(a);

  let a = 10;
}
```

Accessing a before initialization causes an error.

Function Hoisting

Function Declaration

```
sayHello();

function sayHello() {
   console.log("Hello");
}
```

Output:

Hello

Because the entire function is hoisted.

Function Expression

```
sayHello();

var sayHello = function() {
   console.log("Hello");
};
```

Output:

TypeError

Only the variable declaration is hoisted, not the function definition.

Closures

Closures are one of the most powerful concepts in JavaScript.

A closure occurs when an inner function remembers variables from its outer scope even after the outer function has finished executing.

Basic Example

```
function outer() {

   let count = 0;

   function inner() {
      count++;
      console.log(count);
   }

   return inner;
}

const counter = outer();

counter();
counter();
counter();
```

How it Works

Step 1

const counter = outer();

`outer()` executes and returns `inner()`.

Normally local variables should disappear after execution.

Step 2

Because `inner()` still uses `count`, JavaScript keeps it in memory.

Step 3

Every call remembers the previous value.

counter();

Output:

1
counter();

Output:

2

Real Life Example: Private Variables

```
function createBankAccount(initialBalance) {

    let balance = initialBalance;

    return {

        deposit(amount) {
            balance += amount;
            console.log("Balance:", balance);
        },

        withdraw(amount) {
            balance -= amount;
            console.log("Balance:", balance);
        }
    };
}

const account = createBankAccount(1000);

account.deposit(500);
account.withdraw(200);
```

Output:

Balance: 1500
Balance: 1300

Nobody can directly access `balance`.

console.log(account.balance);

Output:

undefined

This is data encapsulation using closures.

Relationship between HOF and Closures

```
function multiplier(factor) {
return function(number) {
return factor * number;
};
}
const triple = multiplier(3);
console.log(triple(10));
```

`multiplier()` is a Higher-Order Function.

Returned function forms a Closure.

`factor` is remembered even after `multiplier()` ends.

Interview Questions

Q1. What is a Higher-Order Function?

Q2. What is a Lambda Function?

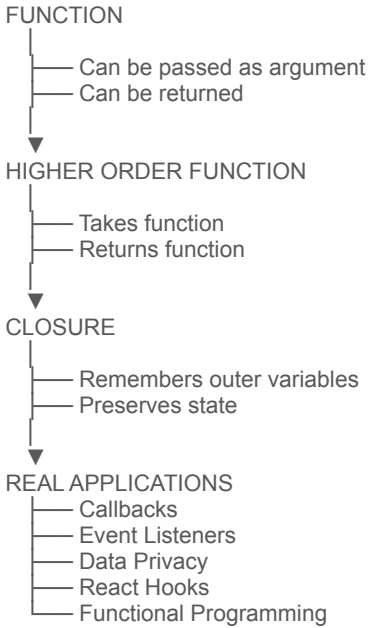
Q3. What is Hoisting?

Q4. What is a Closure?

Q5. Why are Closures Used?

- Data hiding
- Encapsulation
- Private variables
- Function factories
- Event handlers
- Callbacks
- Memoization

Concept Flow diagram



Task

Create reusable functions and closure-based examples

Reusable Functions Topics

Function Reusability Concepts
Writing Generic Functions
Parameterized Functions
Utility Functions
Function Modularization
Function Libraries
Function Composition
Function Factories
DRY (Don't Repeat Yourself) Principle
Reusable Validation Functions
Reusable Calculation Functions
Reusable Formatting Functions
Reusable Data Processing Functions
Reusable Event Handler Functions
Best Practices for Reusable Functions

Closure-Based Example Topics

Introduction to Closures
Lexical Scope and Closures
State Preservation using Closures
Private Variables using Closures
Data Encapsulation with Closures
Function Factories using Closures
Counter Application
ID Generator
Bank Account Manager
Shopping Cart Manager
User Authentication System
Theme/Settings Manager
Countdown Timer
Session Management
Access Control Mechanism
Memoization (Caching)
Rate Limiting
Debouncing
Throttling
Module Pattern using Closures

Interview-Oriented Topics

Closures for Data Hiding
Closures in Event Listeners
Closures in Loops
Closures with Asynchronous Functions
Function Factories vs Closures
Memory Implications of Closures
Practical Uses of Closures in Frameworks
Closures in React Hooks
Common Closure Pitfalls
Optimization Techniques with Closures

Real-World Closure Projects

Visitor Counter
Login Attempt Tracker
Quiz Score Tracker
Cart Item Counter
Expense Tracker
Notification Manager
Dark/Light Theme Switcher
API Request Cache
OTP Generator
Employee ID Generator